jvo, hdaniel, jbarateiro

Lab 4: Rotas

Este lab deve ser feito individualmente.

Submeta o seu código ao mooshak http://deei-mooshak.ualg.pt/~jvo/ usando o seu login da ualg (sem @ualg.pt). Ex: a123456

Uma submissão ao problema permanecerá *pending* até que seja validada pelo professor durante a aula prática. Só as submissões *final* serão consideradas para avaliação.

Todas as submissões deverão ser feitas até

23 de março 2026

A validação poderá ser feita posteriormente, se necessário, até 9 de abril de 2026

NB: Nos problemas seguintes, sempre que necessário, considera-se que dois double d e g são iguais se $|d-g|<1e-9$

Problema I: Rota

Pretende-se implementar uma classe que defina uma rota, dada por um conjunto de pontos ordenados desde o ponto de partida até ao ponto de chegada. Uma rota terá assim pelo menos dois pontos. O percurso entre cada dois pontos da rota é retilíneo. Dois pontos consecutivos da rota devem ser diferentes.
Tendo uma rota definida, pretende-se saber se esta é intersectada por um segmento de reta.

Este programa deve ter como base as classes implementadas nos Labs 1, 2 e 3. Se não tiver, independentemente do mérito da implementação, terá avaliação zero.

Restrições complementares

Nota: Submissões não aceites têm avaliação zero. O não cumprimento de cada uma das restrições abaixo para uma submissão Accepted implica uma redução de 25% na nota da submissão. No limite a nota será zero se todas as restrições não forem cumpridas mesmo que a submissão tenha sido aceite pelo Mooshak.

Seguindo os princípios da programação orientada por objetos, o programa deverá estar estruturado segundo um conjunto de classes apropriadas, nomeadamente:

1) O programa deve adicionar uma classe **Route** que representa as rotas e que implementa uma função que retorna o seu comprimento e o ponto de interseção da rota com um segmento de reta, ou "null" se não a intersectar.

2) Criar testes de unidade para **todos os métodos de todas as classes**.

3) Todas as variáveis de instância das classes têm de ter visibilidade private, sendo acedidas apenas por assessores de leitura. Não podem ter assessores de escrita.
   A classe **Cliente** apenas lê os dados de entrada, processa-os chamando apenas os métodos necessários nas outras classes e escreve o resultado na consola.

4) Nos comentários indique:
   i) Uma linha com a descrição da responsabilidade da classe ou do que o método faz
   ii) @author (apenas para classes)
   iii) @version (apenas para classes; inclua uma data)
   iv) @inv (apenas para classes; inclua uma descrição da invariante usada)
   v) @param (apenas para métodos e construtores)
   vi) @return (apenas para métodos)
   vii) @see (qualquer referência bibliográfica ou sítio web consultado para o desenvolvimento do código respetivo)
   viii) Pré e pós-condições para cada método quando necessário

Entrada

A entrada é composta por 2 linhas.

A primeira linha contém um número par de doubles separados por espaços. Cada par representa as coordenadas cartesianas x e y de cada ponto da rota, ordenados desde a partida até à chegada.

A segunda linha contém 4 doubles, x_A y_A x_B y_B, separados por espaços, que representam as coordenadas cartesianas dos dois Pontos nos extremos A e B do segmento de reta.

Saída

Duas linhas. A primeira com o comprimento da rota, com 2 casas decimais de precisão.
A segunda linha com uma das opções seguintes:

i) As coordenadas do(o) ponto(s) de interseção no formato (x,y) com 2 casas decimais de precisão, ordenados desde o início da rota, ou "null" se não se intersectarem.
ii) Uma mensagem indicando a violação da invariante: "SegmentoReta:iv";

Exemplo de Entrada 1
0 1 1 1 4 4 4 3
2 1 2 4

Exemplo de Saída 1
6.24
(2.00,2.00)

Exemplo de Entrada 2
4 4 6 6 7 7
2 2 4 2

Exemplo de Saída 2
4.24
null

Exemplo de Entrada 3
4 4 4 4 6 7
2 2 4 2

Exemplo de Saída 3
3.61
SegmentoReta:iv

Exemplo de Entrada 4
0 0 2 2 6 2 6 4 11 4
2 1 9 6

Exemplo de Saída 4
13.83
(3.40,2.00) (6.00,3.86) (6.20,4.00)

Problema J: Obstáculos

Tendo uma rota definida, pretende-se saber se esta intersecta um obstáculo.
Os obstáculos são definidos por figuras geométricas: polígonos, triângulos, retângulos, quadrados e círculos. Todas estas figuras, exceto os círculos, são definidas por uma lista ordenada de pontos (vértices), no sentido dos ponteiros do relógio. Dois vértices consecutivos formam uma aresta, sendo a última aresta do polígono composta pela ligação entre o último e o primeiro vértice.

Cada uma destas classes tem a seguinte invariante de instância

- Os polígonos são definidos por uma lista ordenada de pontos (vértices), no sentido dos ponteiros do relógio, em número igual ou superior a 3.
- Nos triângulos os vértices não podem ser colineares.
- Os retângulos têm os lados opostos iguais e as diagonais com o mesmo comprimento.
- Os quadrados têm todos os lados iguais e as diagonais com o mesmo comprimento.
- Os círculos têm o raio maior que zero.

Se as invariantes forem violadas deve ser terminado o programa e impressa a mensagem correspondente à classe: "Poligono:iv", "Triangulo:iv", "Retangulo:iv", "Quadrado:iv" ou "Circulo:iv".

Este programa deve ter como base as classes implementadas nos Labs 1, 2 e 3 e o problema I. Se não tiver, independentemente do mérito da implementação, terá avaliação zero.

Restrições complementares
Nota: Submissões não aceites têm avaliação zero. O não cumprimento de cada uma das restrições abaixo para uma submissão Accepted implica uma redução de 20% na nota da submissão. No limite a nota será zero se todas as restrições não forem cumpridas mesmo que a submissão tenha sido aceite pelo Mooshak.

Seguindo os princípios da programação orientada por objetos, o programa deverá estar estruturado segundo um conjunto de classes apropriadas, nomeadamente:

1) As classes geométricas devem estar estruturadas numa hierarquia coerente e que permita evitar a duplicação de código.

2) Na validação apresente um diagrama UML de todas as classes: Ponto, Vetor, SegmentoReta, AutoPilot, Route, Poligono, Triangulo, Retangulo, Quadrado, Circulo e outras que sejam necessárias.

3) Criar testes de unidade para **todos os métodos de todas as classes**.

4) Todas as variáveis de instância das classes têm de ter visibilidade private, sendo acedidas apenas por assessores de leitura. Não podem ter assessores de escrita.
A classe **Cliente** apenas lê os dados de entrada, processa-os chamando apenas os métodos necessários nas outras classes e escreve o resultado na consola.

5) Nos comentários indique:
   i) Uma linha com a descrição da responsabilidade da classe ou do que o método faz
   ii) @author (apenas para classes)
   iii) @version (apenas para classes; inclua uma data)
   iv) @inv (apenas para classes; inclua uma descrição da invariante usada)
   v) @param (apenas para métodos e construtores)
   vi) @return (apenas para métodos)
   vii) @see (qualquer referência bibliográfica ou sítio web consultado para o desenvolvimento do código respetivo)
   viii) Pré e pós-condições para cada método quando necessário

Entrada

A entrada é composta 2 linhas.

A primeira linha contém um número par de doubles separados por espaços. Cada par representa as coordenadas cartesianas ***x*** e ***y*** de cada ponto da rota.

A segunda linha contém a definição de um obstáculo no seguinte formato. Uma letra indicando a figura e um conjunto de pares de coordenadas x e y que definem os vértices, todos separados por um espaço, conforme o exemplo abaixo:

P 0 0 1 1 2 3 3 4 5 6	(polígono)
S 0 0 0 1 1 1 1 0	(quadrado)
R 0 0 0 2 1 2 1 0	(retângulo)
T 0 0 1 1 2 0	(triângulo)
C 1 2 1	(círculo com centro em 1,2 e raio 1)

Saída

Uma linha com uma das opções seguintes:

i) As coordenadas do(s) ponto(s) de interseção no formato (*x*,*y*) com 2 casas decimais de precisão, ordenados desde o início da rota, ou "null" se não se intersectarem.

ii) Uma mensagem indicando a violação de uma invariante: "SegmentoReta:iv", "Poligono:iv", "Traingulo:iv", "Retangulo:iv", "Quadrado:iv" ou "Circulo:iv".

Exemplo de Entrada 1
2 1 2 4 5 6
P 0 1

Exemplo de Saída 1
Poligono:iv

Exemplo de Entrada 2
2 1 2 4 5 6
T 1 1 2 2 3 3

Exemplo de Saída 2
Triangulo:iv

Exemplo de Entrada 3
2 1 2 4 5 6
R 0 1 2 2 4 1 2 0

Exemplo de Saída 3
Retangulo:iv

Exemplo de Entrada 4
2 1 2 4 5 6
S 0 0 0 1 1 1 2 0

Exemplo de Saída 4
Quadrado:iv

Exemplo de Entrada 5
2 1 2 4 5 6
C 1 1 0

Exemplo de Saída 5
Circulo:iv

Exemplo de Entrada 6
0 0 2 2 2 6
S 1 0 1 3 4 3 4 0

Exemplo de Saída 6
(1.00,1.00) (2.00,3.00)

Exemplo de Entrada 7
2 2 3 4 5 2
T 0 0 1 1 4 1

Exemplo de Saída 7
null